

AutoMap: LLM-Guided Mapping Optimization for DNN Workloads on Heterogeneous Chiplet Systems

Raj Baala
kbaala2@illinois.edu
UIUC

Anand Seetharaman
anandss2@illinois.edu
UIUC

Dennis Binford
dennisb5@illinois.edu
UIUC

Abstract

Chiplet technology has recently gained fast-growing attention as monolithic chips become increasingly expensive to manufacture and approach physical scaling limits. With the slowing of Moore’s Law and advances in packaging technologies such as 2.5D and 3D integration, chiplet-based architectures allow designers to combine dies from different process nodes to improve cost, scalability, and performance for modern AI workloads.

However, chiplet systems introduce a difficult workload-mapping problem. Unlike monolithic accelerators, chiplet-based systems must account for inter-chiplet communication latency, bandwidth limits, and energy costs across the Network-on-Package. Poor DNN layer placement can increase communication overhead, create congestion, and reduce the benefits of heterogeneous compute fabrics.

This paper proposes AutoMap, an LLM-guided agentic framework for optimizing DNN workload mapping on heterogeneous chiplet systems. AutoMap uses simulation feedback from CHIPSIM to analyze bottlenecks, generate improved mapping algorithms, validate generated code, and iteratively refine mappings. The goal is to reduce total inference latency and communication overhead compared to traditional nearest-neighbor mapping baselines.

Source Code: <https://github.com/kbaala2/ECE511-Final-Research-Project.git>

1 Background and Motivation

1.1 Rise of Chiplet Architecture

Chiplet technology has recently gained fast growing attention as monolithic chips are becoming too expensive to manufacture and reaching physical limits. With the end of Moore’s Law and advancement in packaging technology such as 2.5D and 3D stacking, tech leaders such as AMD have switched to a “chiplet” architecture in order to mix-and-match chips of different technology nodes to unlock price and performance gains to accommodate for today’s AI era[3]. Very recent work in the chiplet research field has explored the area of AI for hardware, specifically the use of LLMs for hardware design and optimizations.

1.2 Inter-Chiplet Communication

At the physical level, inter-chiplet communication is governed by emerging standards like the Universal Chiplet Interconnect Express (UCIe), which provide a standardized protocol stack for die-to-die (D2D) links [7]. Unlike monolithic interconnects, these links must balance aggressive bandwidth density requirements with strict energy-per-bit constraints, as moving data across the packaging substrate is significantly more expensive than on-die communication [3].

The shift to 2.5D integration transitions the primary system bottleneck from compute-bound to communication-bound. Recent performance analyses demonstrate that poor workload partitioning can exacerbate Network-on-Package (NoP) congestion, potentially increasing system latency by an order of magnitude [5]. To address this, high-fidelity co-simulation frameworks like CHIPSIM have been developed to accurately model the complex interplay between computation stalls and interconnect contention, providing a necessary foundation for evaluating advanced mapping strategies [6].

2 Introduction

The rise of Large Language Models (LLMs) and generative AI has pushed monolithic silicon design to its physical and economic limits. As traditional scaling fails to keep pace with the memory and compute requirements of trillion-parameter models, **2.5D chiplet-based architectures** have emerged as the standard for high-performance AI accelerators. By decomposing a massive System-on-Chip (SoC) into smaller, specialized dies integrated on an interposer, architects can achieve higher yields and utilize heterogeneous processes. However, this modularity introduces a critical “tax”: the **Network-on-Package (NoP)**. In these systems, moving data across the substrate is significantly more expensive—in both power and latency—than on-die communication. Suboptimal workload partitioning can lead to a “performance cliff,” where interconnect congestion and pipeline stalls account for the vast majority of execution time, effectively nullifying the advantages of the chiplet paradigm.

2.1 Research Problem

Current approaches to chiplet mapping generally fall into two categories: simple greedy heuristics, such as *Nearest Neighbor* placement, or “blind” search algorithms like *Genetic Algorithms (GAs)*. While heuristics are fast, they lack the global visibility required to avoid NoP hotspots in complex DNN workloads. Conversely, while search-based frameworks like *Compass* offer a sophisticated 2D spatio-temporal encoding for mapping, they rely on iterative random mutations that do not possess an inherent understanding of hardware-software co-design. There is a clear research gap for an “intelligent” architect—one capable of reasoning about the structural dependencies of a DNN and the physical constraints of a 2.5D substrate to propose optimized configurations without exhaustive brute-force search.

2.2 Proposed Idea: AutoMap

In this paper, we propose an autonomous, closed-loop framework that utilizes LLM agents to optimize chiplet mapping through high-level architectural reasoning. Unlike traditional optimizers, our

agent ingests a formal representation of the DNN workload—specifically its computational dependencies and activation volumes—alongside the physical metadata of the chiplet fabric. By interpreting these inputs, the LLM generates a structured spatio-temporal schedule that minimizes the communication distance between high-bandwidth layers while maximizing pipeline concurrency. This initial proposal is then verified through cycle-accurate simulation, with the resulting performance stalls fed back to the agent. This allows the LLM to perform “bottleneck attribution,” iteratively adjusting the mapping to resolve interconnect congestion and compute starvation.

2.3 Summary of Contributions

Our primary contributions are as follows:

- **Spatio-Temporal Orchestration Framework:** We introduce a “Determinism-First” pipeline that uses a structured canonical schema to translate LLM-guided mapping decisions into cycle-accurate simulation configurations.
- **Autonomous Bottleneck Attribution:** We develop a closed-loop agentic flow that ingests performance logs from the CHIPSIM co-simulator, allowing the LLM to identify NoP hotspots and iteratively “repair” mapping strategies to minimize overall latency.
- **Comparative Performance Analysis:** We present a comprehensive study evaluating the efficacy of our agent-driven approach against traditional, non-agentic baselines. By benchmarking our results against the standard Nearest-Neighbor strategy utilized in the simulator, we quantify significant improvements in latency across diverse workloads.

3 Design

3.1 Configuration Exploration

CHIPSIM models a heterogeneous chiplet fabric in which individual dies are assigned one of three functional types, each playing a distinct role in DNN inference.

IMC (In-Memory Compute) chiplets perform analog matrix-vector multiplication directly inside memory arrays. They excel at compute-bound DNN layers such as convolutional and fully-connected layers by reducing data movement between compute and memory. However, they have limited flexibility for non-linear operations.

CMOS chiplets are standard digital logic chiplets that implement control, activation functions, pooling, and element-wise operations. They complement IMC chiplets by handling operations that IMC cannot efficiently execute.

I/O chiplets manage off-package communication such as DRAM access and the host interface. They are excluded from DNN layer mapping because they do not perform computation, but they contribute to baseline communication latency when weights or activations must be fetched from off-package memory. In all configurations evaluated in this work, four IO chiplets are fixed at the corners of the 10×10 mesh.

Baseline Model and Parameter Selection. Before running AutoMap, we established a fixed simulation baseline by selecting three CNN models that span a range of architectural complexity:

AlexNet (8 layers, shallow and fast), ResNet50 (54 layers, medium-depth residual network), and VGG16 (16 wide, compute-intensive layers).

CHIPSIM accepts three primary inputs to configure a simulation run: a workload file describing the target DNN model in a layer-wise CSV format (e.g., `workload_alexnet.csv`), where each row encodes a layer’s type, input/output dimensions, filter sizes, and stride lengths; a hardware configuration specified as a YAML file (e.g., `mapping_100_cmos_with_io.yaml`) found in the `configs/` directory of the CHIPSIM repository, which declares each chiplet’s functional type, position in the mesh, memory capacity, and compute throughput; and a mapping function that assigns DNN layers to chiplets at runtime. A simulation is launched via `simulate.py` and driven by a flat configuration whose parameters span four categories. **Workload parameters** identify the DNN model CSV and the `model_definitions.py` file that serves as the single source of truth for layer architectures. **Hardware parameters** specify the chiplet mapping YAML and the adjacency matrix CSV (e.g., `adj_matrix_10x10_mesh.csv`), a 100 × 100 file consumed by the Garnet network simulator to model packet routing across the interposer mesh. **Communication parameters** govern inter-chiplet data transfer, including the choice of network simulator, whether communication is pipelined or non-pipelined, and packet-level settings such as bits per packet and network frequency. **Simulation control parameters** set cycle budgets, caching behavior, deadlock thresholds, and optional post-processing outputs such as power plots and mapping visualizations.

The configuration parameters were set to the values shown in Table 1.

Table 1: Baseline Simulation Configuration Parameters

configuration Parameter	Value	Reasoning
workload	workload_alexnet.csv	Selected CNN model; swapped for resnet50 and vgg16 in other runs
adj_matrix	adj_matrix_10x10_mesh.csv	Standard 100-chiplet 2D mesh; topology held fixed across all runs
chiplet_mapping	mapping_100_cmos_with_io.yaml	CMOS config; swap to mapping_100_with_io.yaml for IMC runs
model_defs	model_definitions.py	Single source of truth for all DNN architectures from CHIPSim; unchanged
clear_cache	false	Reuse cached Garnet results to speed up repeated runs
comm_simulator	Garnet	Only supported option
comm_method	non-pipelined	Empirically ~55× faster to simulate than pipelined on AlexNet
enable_dsent	false	Power estimation not needed for baseline EDP; keeps runs faster
enable_comm_cache	true	Caches Garnet results so reruns skip network simulation
blocking_age_threshold	10	Default value controlling how long a blocked operation waits before being flagged
weight_stationary	true	Standard dataflow for CNN inference; weights stay on chiplets
weight_loading_strategy	all_at_once	Faster than just_in_time on AlexNet (13,335 μ s vs 16,786 μ s)
bits_per_activation	8	8-bit activations for standard precision DNN inference
bits_per_packet	128	Fixed network packet size for the Nol
network_operation_hz	1,000,000,000	1 GHz network frequency; default hardware spec
gem5_sim_cycles	500,000,000	Max simulation cycles; set high enough to never be the limiting factor
gem5_injection_rate	0.0	No synthetic traffic; only real workload traffic used
gem5_ticks_per_cycle	1000	Fixed time resolution; README states do not change
gem5_deadlock_threshold	null	No deadlock timeout enforced
dsent_tech_node	32	32nm node for interconnect modeling; inactive since enable_dsent is false
warmup_period_us	0.0	Single inference workload; no warmup period to filter
run_wkld_agg_comm	false	Supplementary comm analysis not needed for baseline EDP
run_ind_comm	false	Supplementary comm analysis not needed for baseline EDP
run_net_agg_comm	false	Supplementary comm analysis not needed for baseline EDP
generate_plots	true	Generate power and utilization plots for visual analysis
generate_visualizations	false	Mapping visualizations disabled to keep post-processing fast

To get the values that were eventually chosen in Table 1, each parameter in the configuration was swept using AlexNet to do many iterations of the configurations with a small model to allow for many sweeps in a reasonable time frame in order to get a set of configurations that sped up simulation speed so that more LLM iterations could be performed on larger models. Two simulation parameters stood out for a drastic performance improvement in simulation speed. One configuration parameter that led to immense simulation speedup was `comm_method` which was compared between non-pipelined and pipelined communication scheduling. Non-pipelined was selected as it was found to produce 19% lower inference latency on AlexNet (14,225 μ s vs. 17,677 μ s) and was approximately 55× faster in terms of simulation wall clock time. This is important as simulation speed is critical when running many LLM iterations across multiple configurations. Another important configuration parameter was `weight_loading_strategy` which

was compared between `all_at_once` and `just_in_time`. All-at-once was selected as it produced 24% lower weight loading time (13,335 μ s vs. 16,786 μ s) as bulk transfer of all weights before inference begins is more efficient than loading layer by layer when chiplet memory is not a bottleneck.

A key finding from this parameter sweep was that weight loading accounts for over 93% of total inference time across all models and configurations regardless of which parameter combination was used. This confirmed that communication and weight transfer — not compute — are the dominant bottlenecks, and that layer-to-chiplet assignment is the primary optimization target for AutoMap. All remaining simulation parameters were also swept to ensure for a reasonable configuration in terms of simulation speed.

We defined six heterogeneous chiplet configurations to test the generality of AutoMap across diverse hardware substrates that vary the spatial distribution of IMC and CMOS chiplets across the 96 non-IO positions of a fixed 10×10 mesh. All configurations keep IO chiplets at the four corners and hold all simulation parameters constant, ensuring that observed latency differences reflect only the effect of chiplet placement and mapping strategy. The six layouts are:

- **Top-IMC / Bottom-CMOS:** The top five rows are assigned IMC_E chiplets and the bottom five rows are assigned CMOS_Compute chiplets. This partitions the mesh into a contiguous IMC zone near the IO corner chiplets and a CMOS zone further away testing whether IMC proximity to IO reduces weight loading overhead.
- **Left-IMC / Right-CMOS:** IMC chiplets occupy the left five columns and CMOS chiplets occupy the right five columns. This applies the same 50/50 split along the horizontal axis rather than the vertical axis, testing whether the orientation of the IMC/CMOS partition affects inter-chiplet communication patterns under nearest-neighbor and LLM-guided mapping.
- **Center-IMC:** IMC chiplets are clustered in a 6×6 inner block at the center of the mesh, with CMOS chiplets forming the outer ring. Centralizing high-throughput IMC compute minimizes the average hop distance between adjacent IMC chiplets, which is beneficial for sequential layer chains with heavy activation traffic between adjacent compute stages.
- **Checkerboard:** IMC and CMOS chiplets alternate in a checkerboard pattern across the mesh. This maximizes inter-type locality where every IMC chiplet is directly adjacent to a CMOS chiplet testing whether the LLM can exploit fine-grained heterogeneity that nearest-neighbor mapping ignores.
- **IMC-Heavy (75% IMC):** Approximately 75% of compute chiplets are IMC, occupying the top seven rows, with the remaining 25% CMOS in the bottom three rows. This tests an IMC-dominant system and evaluates whether the added analog compute capacity translates to latency improvements when the mapper must route layers across a large IMC region.
- **CMOS-Heavy (75% CMOS):** Approximately 25% of compute chiplets are IMC, confined to the top three rows, with the remaining 75% CMOS below. This tests a CMOS-dominant

system with a small IMC zone and evaluates whether a limited IMC region can still be exploited effectively by an intelligent mapper.

Running AutoMap across all six configurations and three DNN models allows us to assess whether the LLM’s mapping improvements generalize across different spatial arrangements of heterogeneous compute and to identify which chiplet layouts benefit most from intelligent layer placement. A mapper that only performs well on a single layout may simply be exploiting a fixed heuristic. A truly general mapper must adapt its placement strategy to both the chiplet topology and the heterogeneous compute capabilities of each die.

3.2 LLM-Guided Agentic Flow Architecture

Our implementation uses a structured multi-agent LLM pipeline that works with the CHIPSIM simulator to form a closed-loop optimization system we call AutoMap. The central challenge of using an LLM for hardware mapping is ensuring that its outputs are structurally correct, functionally compatible with the simulator, and evaluable against a reproducible baseline. Rather than having the LLM produce static mapping configurations, AutoMap takes a more expressive approach: the LLM generates drop-in replacement code for CHIPSIM’s mapping algorithm itself. This allows the agent to reason about and modify the mapping strategy — including chiplet selection heuristics, allocation ordering, and communication-aware placement logic — rather than being limited to producing a single fixed layer-to-chiplet assignment. The scope of the LLM’s modifications is tightly constrained to the mapping algorithm; all other simulator components, including the chiplet architecture, adjacency matrix, network topology, and compute models, remain unchanged. This isolation ensures that any observed performance difference is attributable solely to the LLM’s algorithmic improvements rather than hardware configuration changes.

To ensure that LLM-generated code is safe to execute within the simulator, AutoMap defines a formal contract that specifies the required function signature, argument types, return type (a four-element tuple of remaining capacity, allocation percentages, failure flag, and failure reason), and prohibited operations such as import statements or modifications to external interfaces. The LLM is instructed to produce outputs that strictly conform to this contract. Before any generated code is forwarded to the simulator, it passes through an AST-based validation stage that programmatically verifies syntactic correctness, exact argument list conformance, return tuple structure, and the absence of disallowed constructs. When validation fails, the specific errors are fed back to the LLM, which is re-prompted to produce a corrected version. This cycle repeats for a configurable number of retries, ensuring that only structurally correct code reaches the simulator.

The LLM agent is provided with structured context each time it is invoked. An analyzer agent receives the full CHIPSIM configuration — including the chiplet mapping specification, topology parameters, communication method, and weight loading strategy — alongside the source code of the current mapping algorithm and the layer partitioning logic. The analyzer identifies specific weaknesses in the existing nearest-neighbor approach for the given configuration, such as its lack of global awareness, its inability to account

for activation transfer volumes between layers, and its indifference to chiplet type heterogeneity in mixed-architecture configurations. This analysis is then passed to an optimizer agent, which generates a complete replacement mapping function that addresses the identified weaknesses while adhering to the contract. The two-agent decomposition separates architectural reasoning from code generation, allowing each agent to focus on its respective task.

Once the generated code passes validation, it is deterministically injected into the simulator’s source. An injection module reads from a pristine backup of the original mapper, updates the default mapping function reference, adds the necessary routing entry so the simulator can locate the new function, and appends the generated method to the class — all without the LLM touching any wiring code. The modified simulator is then executed against the target configuration, and the runtime handler captures the outcome. If the simulation crashes due to a runtime error in the generated code, the full traceback is preserved and fed back to the optimizer, which is re-prompted to fix the issue. This runtime retry loop is independent of the validation retry loop, giving the system two levels of error recovery before advancing to evaluation.

After a successful simulation, the evaluator parses CHIPSIM’s formatted output to extract three performance metrics: total inference latency, compute latency, and communication latency. These are compared against pre-computed baseline results obtained by running the unmodified nearest-neighbor mapper on the same configuration. The evaluator computes per-metric improvement percentages and determines whether the generated mapper outperforms the current best. If an improvement is detected, the new mapper is retained; otherwise, the simulator is reverted to the previous best. The evaluation summary — including which metrics improved, which regressed, and by how much — is injected back into the agent’s context for the next iteration, directing the LLM’s attention toward the highest-impact areas for further optimization.

The core contribution of AutoMap is its iterative refinement loop. After each evaluation, the full optimization history — including prior metrics, outcomes, and the strategies that were attempted — is fed back to the analyzer, which is instructed to propose a different approach than what was previously tried. This history-aware prompting prevents the agent from repeating failed strategies and encourages exploration of the optimization space. The loop runs for a configurable number of iterations or terminates early if three consecutive iterations fail to produce improvement, indicating convergence. Performance metrics are tracked across all iterations to produce optimization trajectories that quantify the agent’s cumulative impact.

All AutoMap results are evaluated against the nearest-neighbor baseline implemented directly in CHIPSIM, which assigns each successive DNN layer to the spatially closest chiplet in a greedy, locally optimal order. Baseline results are pre-computed and stored so that they do not need to be re-run for each experiment, and each agent-optimized iteration produces its own separate results directory for side-by-side comparison. Experiments are conducted across multiple CHIPSIM configurations spanning various heterogeneous chiplet architectures to assess the generality of the agent’s optimizations. All experiments use identical hardware configurations, topology, and workload parameters to ensure that comparisons between the baseline and agent-generated mappers reflect only the

effect of the mapping algorithm. Figure 1 illustrates the high-level agentic flow.

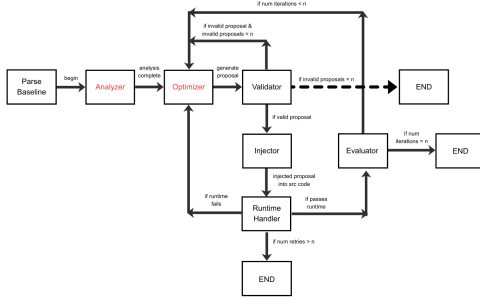


Figure 1: Agentic Flow Graph

4 Implementation

4.1 LLM Integration

AutoMap requires an LLM capable of generating syntactically correct Python functions that adhere to a strict interface contract while reasoning about chiplet architecture constraints, communication topology, and DNN layer characteristics. Over the course of development, we evaluated several model and provider combinations to identify the configuration that best balanced code generation quality, inference speed, and contract compliance.

Initial prototyping was conducted using Llama 3.3 70B served through Groq’s free-tier API, which provided sub-second inference latency due to Groq’s custom LPU hardware. This configuration was sufficient for developing and debugging the agent pipeline, validating the graph structure, and iterating on prompt design. However, Groq’s rate limits (30 requests per minute on the free tier) became a constraint during batch experimentation across multiple CHIPSIM configurations. We also explored local inference using Ollama with Qwen2.5-Coder 32B and Qwen3 32B on an RTX 4090 (24GB VRAM). While this eliminated API dependency and rate limits, the per-call latency of 30 to 60 seconds per invocation — combined with the smaller models’ higher rate of contract violations requiring additional retry cycles — made local inference impractical for the iterative optimization runs the system requires.

For production experiments, we transitioned to DeepSeek V3.2 and DeepSeek V4 Pro accessed through OpenRouter’s unified API. DeepSeek V3.2 is a 671-billion-parameter Mixture-of-Experts model featuring DeepSeek Sparse Attention for efficient long-context processing, with strong demonstrated performance on code generation and agentic tool-use benchmarks. DeepSeek V4 Pro, released in April 2026, scales to 1.6 trillion total parameters with 49 billion activated per token and supports a one-million-token context window. Both models were selected for their code generation quality and their ability to reliably produce outputs conforming to AutoMap’s mapper contract — the structured specification that constrains the LLM’s output to a valid drop-in function for CHIPSIM’s mapping algorithm.

OpenRouter serves as an API routing layer that provides a unified OpenAI-compatible interface across multiple model providers. This allowed us to switch between **DeepSeek V3.2** and **DeepSeek V4 Pro** — as well as any previously evaluated model — by changing a single model identifier string in the agent configuration. The LangChain framework’s provider abstraction further simplifies this: the LLM object used by all agent nodes is instantiated once and passed uniformly throughout the pipeline, making model selection independent of the agent architecture.

The mapper contract itself is a structured prompt component that accompanies every optimizer invocation. It specifies the exact function signature the LLM must produce (including argument names, order, and default values), the required four-element return tuple structure, the helper methods and data accessors available within the CHIPSIM mapper class, and explicit prohibitions against import statements or modifications to external interfaces. This contract acts as the primary mechanism for grounding the LLM’s output in a representation that is both structurally valid and executable within the simulator.

4.2 Agent Pipeline

The AutoMap agent pipeline is implemented as a directed graph using LangGraph, a framework for building stateful, multi-step LLM applications with explicit control flow. LangGraph was selected over alternatives such as CrewAI for its support of conditional branching, typed state dictionaries, and cyclic graph structures — all of which are essential for the iterative optimization loop with multiple error recovery paths that AutoMap requires.

The pipeline is composed of seven nodes connected by conditional edges, with a shared typed state dictionary that flows through the entire graph. The state carries all data accumulated across the pipeline: the CHIPSIM configuration context, the analyzer’s assessment, the optimizer’s generated code, validation results, injection status, simulation output, parsed metrics, baseline and best-known metrics, per-iteration improvement percentages, the full optimization history, and iteration control variables.

Execution begins at the **baseline** node, which loads pre-computed performance metrics for the unmodified nearest-neighbor mapper from a designated results directory. This avoids re-running a simulation that has already been completed and establishes the reference point against which all subsequent iterations are measured. If the baseline metrics cannot be loaded, the pipeline terminates immediately.

The **analyzer** node reads the original mapping algorithm source code and the layer partitioning logic, along with the CHIPSIM configuration, and invokes the LLM to identify specific weaknesses in the nearest-neighbor approach for the given chiplet architecture. On subsequent iterations, the analyzer also receives the full optimization history — including which strategies were previously attempted and their outcomes — and is explicitly instructed to propose a different approach than what was tried before. This history-aware prompting prevents the agent from repeating failed strategies across iterations.

The **optimizer** node receives the analyzer’s assessment, the original mapper source code for reference, and any error context from prior attempts. It invokes the LLM with the mapper contract

to generate a complete drop-in replacement function. The optimizer's prompt is designed to handle two distinct error recovery scenarios: if the previous attempt failed validation, the specific validation errors are included so the LLM can correct structural issues; if the previous attempt failed at runtime, the full traceback is preserved and included so the LLM can fix logical errors such as incorrect attribute access or missing null checks. A field called *last_runtime_error* is maintained across validation retries so that if a runtime failure triggers a new proposal that then fails validation, the subsequent correction still has access to the original runtime traceback.

The **validator** node performs AST-based verification of the generated code without executing it. It parses the output into an abstract syntax tree and programmatically checks that exactly one function definition exists, that the function name begins with an underscore, that the argument list matches the expected signature exactly, that the last two arguments default to None, that at least one return statement produces a four-element tuple, and that no import statements are present. When validation fails, the specific errors are returned to the optimizer for correction. This cycle repeats for a configurable number of retries (defaulting to three) before the pipeline gives up on the current iteration.

Upon successful validation, the injector node deterministically integrates the generated function into CHIPSIM's mapper source file. It reads from a pristine backup of the original file, updates the default mapping function reference in the class constructor, adds a routing entry in the function dispatcher so the simulator can locate the new method by name, and appends the indented function body to the class. All of this wiring is performed programmatically — the LLM never modifies any code outside of the mapping function it generates.

The **runtime handler** node executes the modified simulator as a subprocess, captures the outcome, and routes accordingly. If the simulation succeeds, control passes to the evaluator. If a runtime error occurs and retries remain, the traceback is fed back to the optimizer for a corrected proposal. If runtime retries are exhausted, control still passes to the evaluator, which records the failure in the optimization history.

The **evaluator** node parses CHIPSIM's formatted output file to extract three performance metrics: total inference latency, compute latency, and communication latency. It computes per-metric improvement percentages against the baseline and determines whether the current iteration outperforms the previous best based on total latency as the primary metric. If an improvement is detected, the new mapper is retained as the current best. If not, the simulator is reverted to the previous best mapper. The evaluation summary — including which metrics improved and which regressed — is injected into the state so that the next iteration's analyzer and optimizer can use it as feedback. The loop terminates when the maximum iteration count is reached or after three consecutive iterations fail to produce improvement.

Each iteration produces its own separate results directory within CHIPSIM's output structure, named by the configuration and function name (Ex. *resnet50_hetero_checkerboard_improved_mapper*). This enables side-by-side comparison of simulation outputs across iterations and against the baseline without any results being overwritten.

The pipeline is parameterized through command-line arguments that specify the CHIPSIM configuration file, the maximum number of optimization iterations, and the retry budgets for validation and runtime errors. A batch execution script enables unattended runs across multiple configurations, logging each run's output and printing a consolidated summary upon completion. Table 2 gives a summary of the purpose of each node in the graph.

Table 2: Mapping Optimizer Graph Nodes

Node	Description
Baseline	Extracts baseline latency results from baseline results folder.
Analyzer	Analyzes the input of the given configuration file and generates a mapping suggestion based on the configuration and bottleneck of the default nearest neighbor.
Optimizer	Ingests feedback from the Analyzer and generates a new mapping algorithm. Uses hard constraints as inputs in order to keep the same formatting that simulator expects: return vals, function args, helper apis, etc.
Validator	Does lint checking to ensure that the Optimizer generated code that obeyed the given constraints.
Injector	After validation of generated code, injects all generated code directly into the necessary src files to properly modify the mapping configuration.
Runtime Handler	Runs the simulation with injected changes and collects logs, errors, and performance results. Either feeds information back into Optimizer or moves to evaluator
Evaluator	Compares results to best performing configuration and returns feedback for the next iteration

4.3 Simulator Integration

AutoMap interfaces with CHIPSIM through three integration points: source code injection, simulation execution, and results parsing.

The source code injection targets a single file in the CHIPSIM source tree — the model mapper module that implements the layer-to-chiplet assignment algorithm. A pristine copy of this file is created as a backup on the first run and is never modified. Every injection starts from this backup, ensuring that successive iterations do not accumulate edits. The injection process modifies three specific locations in the file: the default mapping function parameter in the class constructor, the function dispatcher that routes the mapping function name to the corresponding method, and the class body where the new method is appended. The function dispatcher is updated by scanning line-by-line for the existing routing chain and inserting the new entry at the correct position, which is robust to prior injections or manual edits in the file. After injection, Python's bytecode cache is cleared to ensure the simulator loads the modified source.

Simulation execution is handled through subprocess invocation from the agent's working directory. An environment variable (*CHIPSIM_RESULTS_DIR_NAME*) is set before each run to control the output directory name, replacing CHIPSIM's default timestamp-based naming with a predictable label derived from the configuration name and the generated function name. This enables the evaluator to locate the correct results file without scanning the filesystem. Previous results for the same label are removed before each run to ensure that the evaluator always reads fresh output.

Results parsing reads from CHIPSIM's post-processed output file, specifically the Approach Comparison Metrics report that the simulator's output manager generates after each run. This file contains pipe-delimited tables with per-model average latencies across three categories. The parser identifies each table by its header text, extracts the model name and the main simulation metric value from each data row, and returns a structured dictionary with per-model and aggregate metrics. The same parser is used for both baseline loading (reading from a pre-computed results directory maintained separately from the agent's output) and per-iteration evaluation (reading from the CHIPSIM results tree), ensuring consistent metric extraction across all comparisons.

5 Evaluation

Our results proved that an agentic framework can be a great method for efficiently mapping DNN workloads to chiplets. However, the type of workload and LLM model can have a large impact in its effectiveness.

Overall, the optimizations mostly improved latency by reducing communication latency, not compute latency. From our results in nearly every successful case, total latency tracks communication latency closely because communication dominates the runtime.

The strongest improvements came from mappers that changed placement behavior from the simple nearest neighbor expansion to more topology-aware, communication-aware, or multi-seed/load-balanced placement.

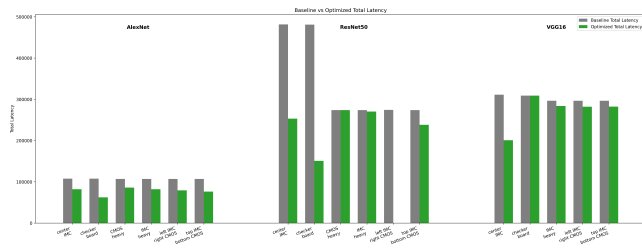


Figure 2: Baseline vs Optimized Total Latency Deepseek v3.2

The v3.2 and v4 optimization results show that the agentic mapper primarily improves performance by reducing communication latency. Across most configurations, communication latency dominates total latency, so even mappings that increase compute latency can still improve end-to-end performance when they significantly reduce inter-chiplet data movement. In v3.2, the largest reported improvement was *resnet50_hetero_left_imc_right_cmos*, which reduced total latency by 99.94%; however, this result is an outlier and should be manually verified. Other strong v3.2 results included

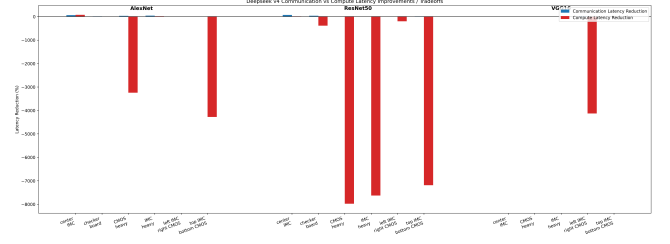


Figure 3: Compute vs Communication Tradeoff Deepseek v4

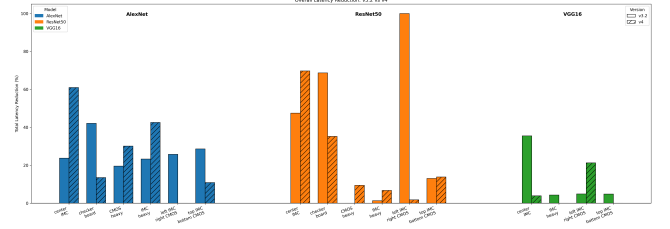


Figure 4: Overall Latency Reduction Deepseek v4 vs Deepseek v3.2

resnet50_hetero_checkerboard and *resnet50_hetero_center_imc*, which improved by 68.66% and 47.46%, respectively. In v4, the strongest improvement was *resnet50_hetero_center_imc*, which improved by 69.69%, followed by *alexnet_hetero_center_imc* at 60.94% and *alexnet_hetero_imc_heavy* at 42.57%.

Comparing the two versions, v4 improved several AlexNet configurations and achieved the best non-outlier ResNet50 center-IMC result, but it was not uniformly better than v3.2. Some configurations regressed, especially *resnet50_hetero_checkerboard*, which dropped from 68.66% improvement in v3.2 to 35.24% in v4. VGG16 also remained difficult to optimize consistently, with several v4 configurations showing no improvement. These results suggest that the effectiveness of the optimizer depends heavily on the interaction between workload structure and chiplet topology. The best improvements occur when the mapper can reduce communication between dependent layers without creating excessive compute imbalance or routing congestion.

The LLM changed performance because each generated mapper changed the placement policy used to assign DNN layers to chiplets. Since communication latency dominated total latency, performance improved when the LLM found mappings that reduced data movement between dependent layers. However, some mappings increased compute latency or failed to exploit the chiplet topology, causing smaller improvements or regressions. Therefore, the LLM's impact depended on whether its generated heuristic matched the workload's communication pattern and the architecture's physical layout.

6 Related Work

6.1 LLMs for Chip Design

The use of LLMs in hardware design opens up opportunities for fast design space exploration, significantly reducing the time required in the architect's design process. Recent research has demonstrated the

effectiveness of this approach through successful CPU tapeouts that used LLM-generated HDL code. However, despite these successes, LLMs remain error-prone and can generate incorrect HDL that does not meet the given specification. As a result, skilled hardware engineers are still needed to verify generated designs and ensure reliability [1].

To further unlock the benefits of LLMs for chip design, engineers have integrated LLMs into feedback-loop architectures that reduce the amount of manual prompting required. In these systems, the LLM continuously ingests previous outputs as new input and uses that context to determine the next best course of action. This improves productivity and leads to stronger Pass@k metrics [9].

More recent advancements have extended this idea by deploying multiple autonomous LLM agents to address chiplet design. In these systems, each LLM agent is assigned a specific task within the design pipeline, improving the quality of decisions generated at each stage. This transition from conversational assistants to autonomous agents reflects a broader shift in Electronic Design Automation (EDA), where LLMs are no longer limited to producing isolated code snippets but can help manage end-to-end design toolchains [2].

For example, the MAHL framework uses a hierarchical multi-agent system to decompose complex AI algorithms into structured hardware layers. This helps ensure that generated chiplet modules remain modular and reusable, which is especially important for 2.5D integration where flattened LLM-generated designs often fail to meet modularity requirements [8].

LLMs are also being applied beyond hardware description and into high-level architectural optimization. Recent work such as LUMINA demonstrates that LLM agents can interpret simulation logs, perform bottleneck analysis, and act as a “Smart Architect” to guide accelerator architecture exploration. This allows LLM-based systems to provide reasoning capabilities that can exceed traditional search-based heuristics [10].

To ensure that these advancements translate into measurable improvements in Power, Performance, and Area (PPA), researchers have introduced standardized benchmarks such as RTLMM. These benchmarks provide a rigorous framework for evaluating LLM-generated designs against human-expert baselines and quantifying the quality of the generated silicon [4].

7 Conclusion

The mapping of DNN workloads onto heterogeneous chiplet fabrics is an increasingly critical problem as monolithic accelerator designs reach their physical and economic limits. Poor layer-to-chiplet assignment creates Network-on-Package communication bottlenecks that nullify the performance advantages of chiplet integration, yet existing approaches — greedy nearest-neighbor heuristics and search-based genetic algorithms — lack the architectural reasoning needed to avoid these bottlenecks in complex inference workloads.

This paper presented AutoMap, an LLM-guided agentic framework that addresses this problem by replacing CHIPSIM’s static nearest-neighbor mapper with iteratively refined, LLM-generated

mapping algorithms. AutoMap operates as a closed-loop multi-agent pipeline: an analyzer agent identifies weaknesses in the current mapping strategy for a given chiplet topology and DNN workload, an optimizer agent generates a drop-in replacement mapping function constrained by a formal interface contract, and a validator-injector-runtime chain executes the generated code in CHIPSIM and feeds back performance results for the next iteration. History-aware prompting prevents the LLM from repeating failed strategies and directs optimization effort toward the highest-impact areas of the mapping space.

Evaluated across three CNN models — AlexNet, ResNet50, and VGG16 — on six heterogeneous chiplet configurations combining IMC and CMOS chiplets on a 10×10 mesh, AutoMap consistently reduced total inference latency relative to the nearest-neighbor baseline using both DeepSeek V3.2 and DeepSeek V4 Pro. Improvements were driven almost entirely by reductions in communication latency, confirming that inter-chiplet data movement is the dominant bottleneck in these systems and that layer-to-chiplet assignment is the primary lever for optimization. The strongest gains were achieved on ResNet50 and AlexNet on center-IMC configurations, with improvements exceeding 60% in the best cases. VGG16 proved more resistant to optimization due to its wide layer structure demanding simultaneous use of many chiplets, and results varied between the two LLM backends, indicating that the effectiveness of the optimizer depends on the interaction between model architecture, chiplet topology, and the reasoning capability of the underlying language model.

These results demonstrate that LLM-guided agentic mapping is a viable and effective alternative to static heuristics for chiplet workload placement, capable of identifying topology-aware placement strategies that greedy approaches cannot. At the same time, they highlight the sensitivity of performance gains to workload characteristics and LLM capability, pointing toward the value of stronger models, more optimization iterations, and energy-aware objective functions as natural directions for future work.

8 Future Work

Several extensions can improve AutoMap’s effectiveness and scope.

8.1 Multi-Objective Optimization

Currently AutoMap optimizes only for total inference latency. Real chiplet systems must also balance energy consumption and energy-delay product (EDP). A natural extension is to modify the evaluator to report energy estimates from CHIPSIM (if available) or to model energy via per-chiplet power tables. The agent’s prompt would then instruct it to minimize a weighted sum of latency and energy, or to generate a Pareto frontier of non-dominated mappings.

8.2 Larger and Irregular Topologies

Our fixed 10×10 grid may under-constrain the mapping problem for deep but narrow models like VGG16. Larger chiplet counts (e.g., 20×20) or irregular mesh/ring topologies would create more complex communication bottlenecks. We hypothesize that AutoMap’s advantage over nearest-neighbor will grow as the topology becomes less regular and the number of chiplets increases relative to the number of DNN layers.

8.3 Cross-Workload Generalization

The analyzer currently resets for each new DNN model and chiplet layout. A memory-augmented agent could store successful mapping strategies (e.g., “for residual networks, co-locate skip-connection layers”) and reuse them when a similar workload is detected. This would reduce the number of iterations required per configuration and could be implemented using a vector database of past optimization trajectories.

8.4 Better Runtime Error Recovery

Many runtime failures occurred because the LLM made incorrect assumptions about CHIPSIM’s internal data structures. Fine-tuning a small LLM on CHIPSIM’s codebase (e.g., using LoRA) could teach the optimizer which chiplet attributes, helper methods, and state variables are safe to access. Alternatively, a sandbox execution environment that tests generated mappers on a tiny DNN fragment before full simulation would catch many errors early.

8.5 Evaluation on Transformer Models

Our experiments focused on CNNs (AlexNet, ResNet50, VGG16). Modern generative AI workloads are dominated by Transformers (e.g., BERT, GPT), which have very different dependency patterns—namely, dense attention matrices and long-range residual flows. Evaluating AutoMap on transformer-based DNNs would test whether the LLM can generalize its reasoning to attention-based communication patterns.

References

- [1] Ritvik Choudhary and Daisuke Kawahara. 2023. Chip-Chat: Challenges and Opportunities in Conversational Hardware Design. In *2023 IEEE/ACM International Conference on Computer Aided Design (ICCAD)*. 1–6. doi:10.1145/3670474.3685964
- [2] Zhuolun He, Yuan Pu, Haoyuan Wu, Tairu Qiu, and Bei Yu. 2025. Large Language Models for EDA: Future or Mirage? *ACM Trans. Des. Autom. Electron. Syst.* 30, 6, Article 90 (Oct. 2025), 53 pages. doi:10.1145/3736167
- [3] Gabriel H. Loh and Rajasekaran Swaminathan. 2023. The Next Era for Chiplet Innovation. *2023 Design, Automation & Test in Europe Conference & Exhibition (DATE)* (2023), 1–6. <https://api.semanticscholar.org/CorpusID:259028575>
- [4] Yao Lu, Shang Liu, Qijun Zhang, and Zhiyao Xie. 2023. RTLLM: An Open-Source Benchmark for Design RTL Generation with Large Language Model. arXiv:2308.05345 [cs.LG] <https://arxiv.org/abs/2308.05345>
- [5] Peter Mbua, Peter Forcha, and Christophe Bobda. 2026. Effects of Poor Workload Partitioning on System Performance for Chiplet-Based Systems. *Electronics* 15, 6 (2026). doi:10.3390/electronics15061139
- [6] Lukas Pfromm, Alish Kanani, Harsh Sharma, Janardhan Rao Doppa, Partha Pratim Pande, and Umit Y. Ogras. 2025. CHIPSIM: A Co-Simulation Framework for Deep Learning on Chiplet-Based Systems. *IEEE Open Journal of the Solid-State Circuits Society* 5 (2025), 410–423. doi:10.1109/ojsscs.2025.3626314
- [7] Debendra Das Sharma, Thomas Coughlin, and Tom Coughlin. 2024. Universal Chiplet Interconnect Express: An Open Industry Standard for Memory and Storage Applications. *Computer* 57, 1 (Jan. 2024), 75–81. doi:10.1109/MC.2023.3318769
- [8] Jinwei Tang, Jiayin Qin, Nuo Xu, Pragnya Sudershan Nalla, Yu Cao, Yang, Zhao, and Caiwen Ding. 2025. MAHL: Multi-Agent LLM-Guided Hierarchical Chiplet Design with Adaptive Debugging. arXiv:2508.14053 [cs.AR] <https://arxiv.org/abs/2508.14053>
- [9] Shailja Thakur, Jason Blocklove, Hammond Pearce, Benjamin Tan, Siddharth Garg, and Ramesh Karri. 2024. AutoChip: Automating HDL Generation Using LLM Feedback. arXiv:2311.04887 [cs.PL] <https://arxiv.org/abs/2311.04887>
- [10] Tao Zhang, Rui Ma, Shuotao Xu, Yongqiang Xiong, and Peng Cheng. 2026. LUMINA: LLM-Guided GPU Architecture Exploration via Bottleneck Analysis. arXiv:2603.05904 [cs.AR] <https://arxiv.org/abs/2603.05904>